Auditable concept screening for bioresorbable vascular devices

A domain-tuned 4B model inside SciTag's human-gated multi-agent harness — Adaption sponsor track

TL;DR — We turned a 1,392-row clinical-engineering dataset into a resident 4B domain model via Adaption's Adaptive Data + AutoScientist, then plugged it into SciTag as an OpenAI-compatible agent that calls a real analysis tool before answering. Same data, stronger LoRA: the platform's held-out preference win rate went from **51.67%** → **90.67%**. Every answer carries tool provenance (domain, command, exit code, SHA-256 data hashes), so a human owner can audit the number, not just the prose.

---

1. Problem

Bioresorbable vascular intervention forces a concept decision before any design work: a **drug-eluting stent** (local drug, preserve flow) versus an **occluder / plug** (permanent occlusion, e.g. tumor-feeding artery embolization). Clinicians input a case (lesion size, target vessel, lesion length, blood pressure, intent); the system must return a *quantified* concept screen and a material shortlist — not a generic LLM opinion.

Two hard constraints: (1) numbers must come from physics/tooling, not language-model memory; (2) a human must be able to audit the recommendation.

2. System: SciTag + Adaption

SciTag (zero-dependency Node ≥18) is a research-collaboration harness: a PI dispatches work, agents mount domain packages (knowledge base, skills, data, declared analysis tools), report to their human owner for approval, and only then publish. The platform records tool runs and provenance for every agent answer.

Domain package `vascular-stent` exposes a zero-dependency Python tool (`stent_tool.py`) declared in `domain.json`: material comparison, thin-ring stiffness, Poiseuille baseline, numerical self-checks, and a `concept` command that computes: drug-payload budget vs oral dose, static pressure load and required sealing force for a defect, and material shortlists with degradation-class screening.

Adaption pipeline — `1,392` instruction rows (materials facts, drug budgets, seal loads, case→concept, tool usage, refusal/red-line cases) → Adaptive Data → AutoScientist (Qwen3.5-4B, LoRA). A held-out 144-row split is never trained on.

3. Results

Run	Data	LoRA / schedule	Platform win rate
v1	1,392 rows	r=16, 1 epoch, lr 5e-5 (platform default)	**51.67%** (≈ tie with base)
v2	1,271 rows + reasoning traces	r=16, 1 epoch	40.43% (negative — reasoning style hurt the judge)
v1-strong	same 1,392 rows	**r=32, 3 epochs, lr 1e-4**	**90.67%** (target 0.65 exceeded)

Key finding: **dataset style was not the bottleneck — training strength was.** With identical data, moving from 1 epoch/r16 to 3 epochs/r32 lifted the win rate by ~39 points. The v2 "reasoning-trace" corpus is kept as a documented negative result.

4. Deployment & auditable demo

- LoRA merged into base weights (manual key remap; PEFT injection silently failed under transformers 5.8), served as an **OpenAI-compatible** endpoint (`/v1/chat/completions`), 4-bit NF4, ~3.2 GB VRAM, ~5 tokens/s on an RTX

5060 Laptop.
- SciTag agent `支架适配模型` mounts `vascular-stent` and uses a **forced pre-flight tool call**: the case parameters are parsed, `concept` is executed, and the real JSON is placed in a lean context before generation (also avoids 8 GB VRAM OOM on long contexts).
- Example case (hepatic artery, 4.0 mm lesion, 3.0 mm vessel, 100 mmHg, permanent embolization):

> A (drug-eluting) **not viable**: local payload ~270 µg ≈ 1.39×10^{-5} of a 20 mg/day oral dose — 3+ orders of magnitude short. B (occlusion) **viable**: static load 167 mN → required sealing force **0.335 N** at safety factor 2. Fe / Zn / Mg frames pass the 1.5× margin screen for permanent occlusion. Audit record: `domain=vascular-stent`, `command=concept`, `exitCode=0`, `analysisRuns=1`, data/script SHA-256 in provenance.

5. Reproducibility

- Dataset generator + held-out split: `_adaption/dataset/build_dataset.py` (seed 20261020), dataset card included.
- Training: `upload_and_train*.py` (Adaption SDK 0.15.0); run IDs and win rates logged in `train-state*.json`.
- Serving: `serve_adapted.py` (tools + `<tool_call>` parsing); merging: `merge_lora_stream.py`.
- Harness: `scitag` repo with new tests `test-llm-tools.mjs` (10 tests covering pre-flight, lean tool round, rejection paths) and full `npm test` green (119 regression + suites).
- Repo: `https://github.com/yinkesi/scitag` · Video: `<link>` · Dataset: see `submission/` and notes.

6. Limitations & safety

- The tuned model is a **domain assistant**, not a clinician: no diagnosis, no device sizing decisions; owner approval is mandatory.
- Training data is synthetic/templated from domain KB + published material properties (MatWeb, internal use only — raw CSVs are **not** redistributed).
- Material library lacks corrosion kinetics/fatigue data; all shortlists are order-of-magnitude screens requiring bench/animal validation.
- 4B model still occasionally adds out-of-library materials; core numbers are tool-derived and auditable.

7. Why this fits the sponsor track

Adaption supplied the full loop — data adaptation, automated LoRA search, downloadable weights — and the result is measurable (51.67% → 90.67%) and deployable locally (no API dependency, 3.2 GB VRAM). SciTag supplies the missing half for science: domain tools, human gating, and provenance. Together they produce a **small, auditable, offline domain model** that a research group can run on a laptop.